



Pro-code Statistical Visualization with Seaborn

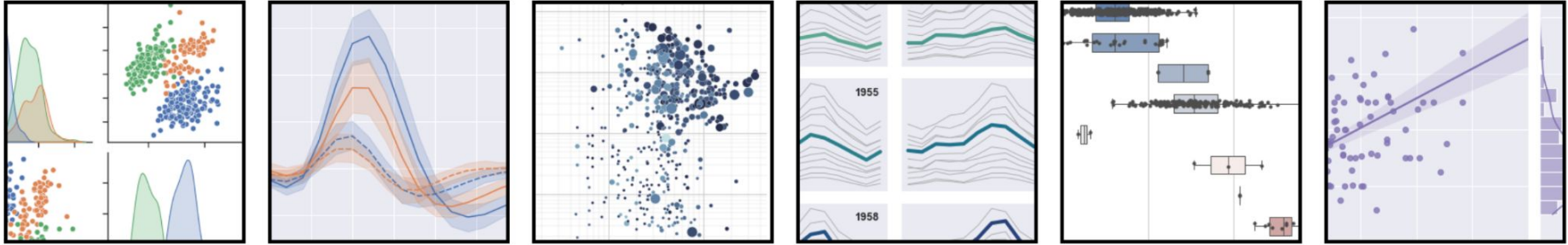
DSCI 454
Week 10, Lecture 1



What are we learning this lecture?

- Pro-code Statistical Visualization with Seaborn
 - Setup
 - Datasets
 - Long-form vs. wide-form data
 - KDE
 - Bootstrapping for Confidence Areas
 - Types of plots

seaborn: statistical data visualization



Seaborn is a Python data visualization library based on [matplotlib](https://matplotlib.org/). It provides a high-level interface for drawing attractive and informative statistical graphics.

seaborn Setup



seaborn is already installed in edStem. It is commonly aliased as **sns** ([why?](#)).

set_theme() is a method that has multiple optional kwargs (all w/ sensible default values) so you can call it by itself or override to your preference:

Params:

context – Scaling parameters: “paper”, “notebook” (default), “talk”, “poster” see :func:`plotting\_context`.

style – Axes style parameters: “white”, “dark”, “whitegrid”, “darkgrid” (default), “ticks”; see :func:`axes\_style`.

palette – Color palette, see :func:`color\_palette`.

And others 170+, “deep” (default)

```
import seaborn as sns
```

```
sns.set_theme(context='poster',  
style='whitegrid', palette='colorblind')
```

seaborn datasets



`get_dataset_names()` returns a list of built-in datasets, each of which you can retrieve as a pandas Dataframe with `load_dataset()`

Unfortunately, `load_dataset()` doesn't work in edStem due to what appears to be a weird proxy issue. It does work in Google Collab and locally via PyCharm, VS Code, etc. So you can:

1. load locally or in Collab,
2. get the dataframe,
3. save to local csv via pandas `to_csv()` (basically the inverse of `read_csv()`), then
4. upload to your edStem workspace,
5. Use pandas `read_csv()` like you're used to.

This process will already be done for you for HWs in edStem but you can do this yourself for any seaborn built-in dataset you want to use in edStem later on.

...previous code here...

```
datasets =sns.get_dataset_names();  
print(*datasets)
```

```
cars=sns.load_dataset('mpg')
```

```
cars.info()  
cars.describe()
```

```
cars.to_csv('mpg.csv', index=False)
```

```
cars=cars.drop_duplicates().dropna()  
print(cars.shape)
```



histplot()

Seaborn can plot a histogram with the **histplot()** function

The first parameter is which feature/column/series you want to plot

Since seaborn is built on top of matplotlib, seaborn functions return matplotlib objects. In this case `kdeplot()` returns a matplotlib Axes object, `ax`. We can call **get_figure()** to get the matplotlib Figure object, and then call `savefig()` just like before to save a .png image file. Use the optional **bbox_inches** kwarg w/ the value 'tight' if you get truncation or clipping.

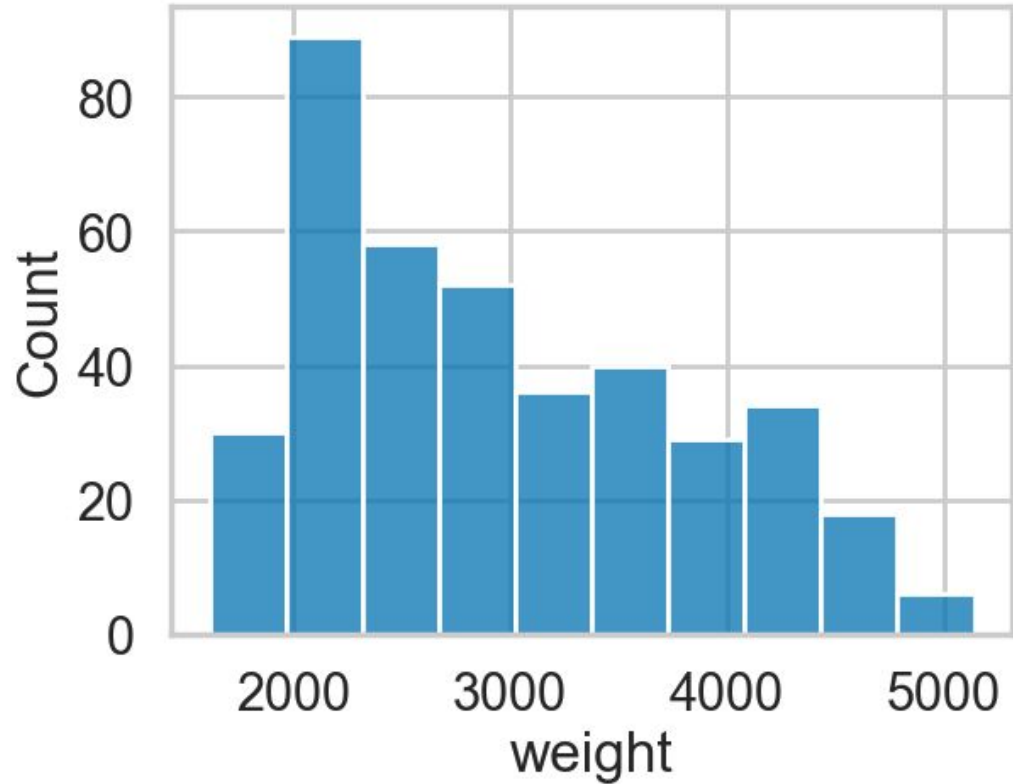
```
ax = sns.histplot(cars.weight)
ax.get_figure().savefig('hist.png',
bbox_inches='tight');
```



histplot() output

Note how seaborn automatically adds axes labels based on the pandas series/column/feature you plotted and the plot type.

Q: What is this missing?



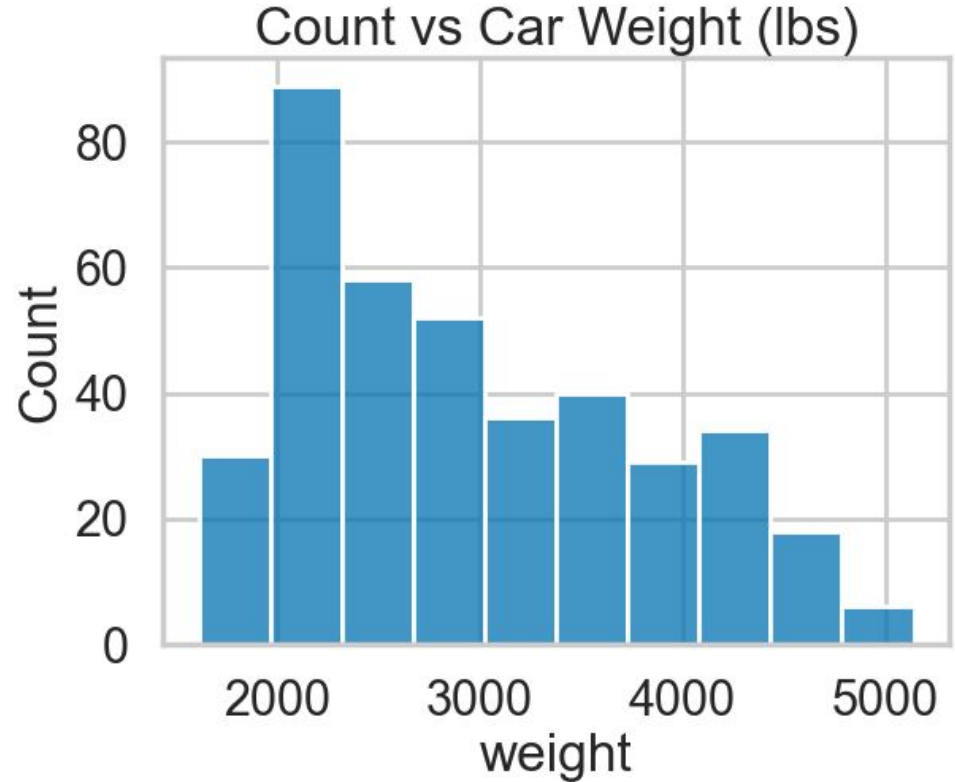


histplot() output w/ Title

```
ax.set_title('Count vs Car  
Weight (lbs)');
```

You can always use all the matplotlib functions and methods you already know!

Note we added unit of measure!
At a minimum, this is what you should be turning in for points



Hue is basically pandas groupby() and move_legend()



Many seaborn charts have an optional kwarg **hue** which takes another feature/series/column and will usually create multiple plots on the same axes grouped by that feature

When an axes gets busy with many plots, it can help to move the legend off the axes via **move_legend()**

Note the explicit kwarg **x** here. This is required if you get a `ValueError: The following variable cannot be assigned with wide-form data: `hue``

```
ax = sns.histplot(x=cars.weight,  
hue=cars.origin)
```

```
ax.set_title('Histogram of Cars  
Gross Weight (lbs) by Origin')
```

```
sns.move_legend(ax, "upper left",  
bbox_to_anchor=(1, 1));
```

```
ax.get_figure().savefig('hist_KDE  
_hued_by_origin.png',  
bbox_inches='tight');
```

Hue and move_legend() output

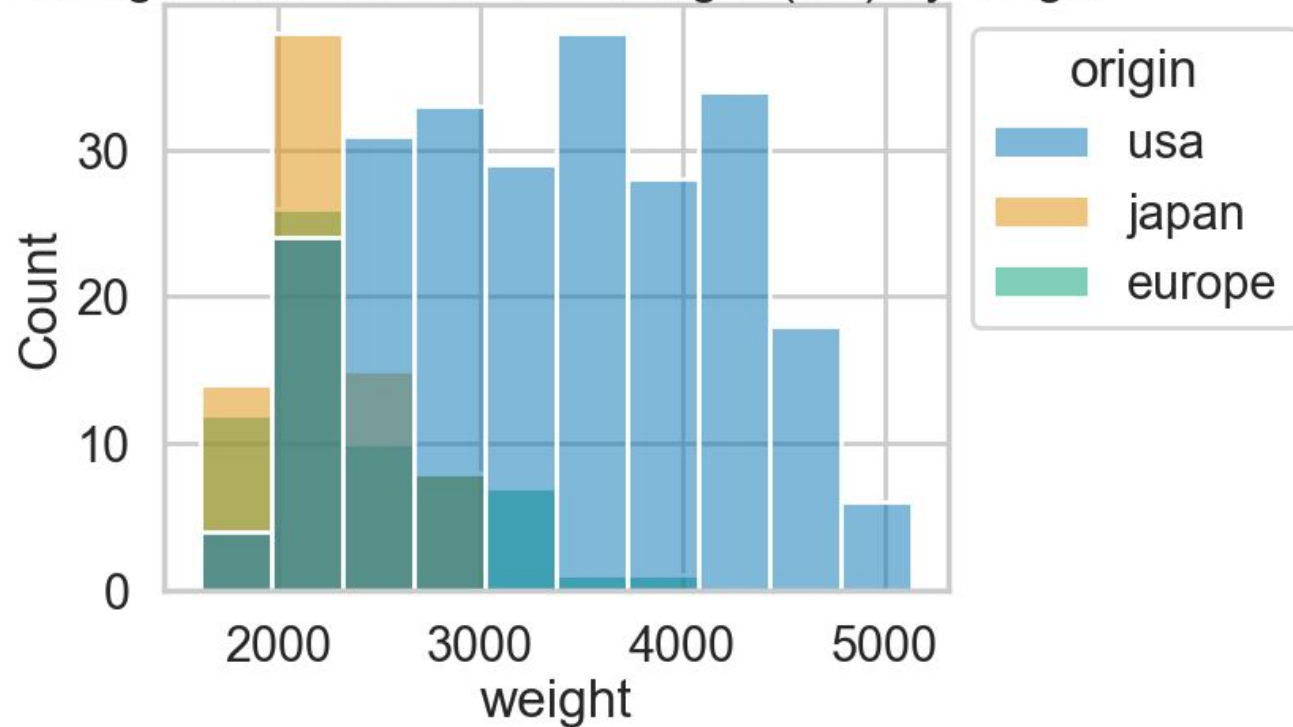


Note alpha (transparency) is enabled by default here

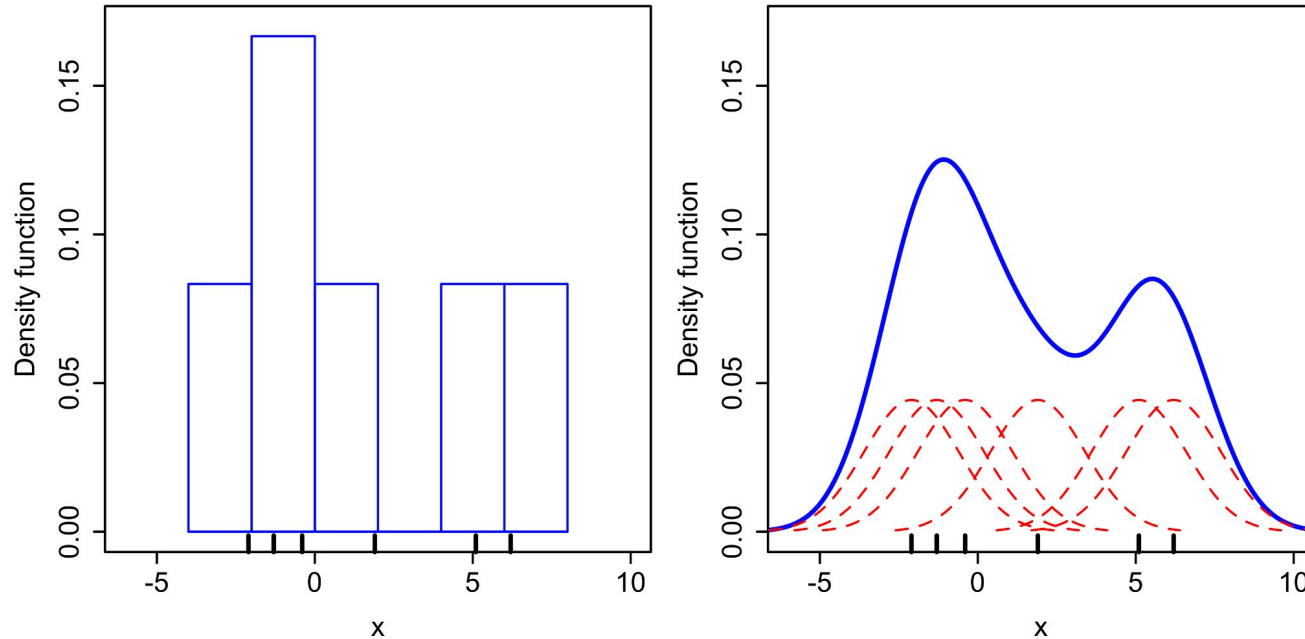
Note this is pretty busy (lower clarity, higher cognitive load)

We will see another way to visualize this same information on separate axes later on

Histogram of Cars Gross Weight (lbs) by Origin



KDE (Kernel Density Estimation)



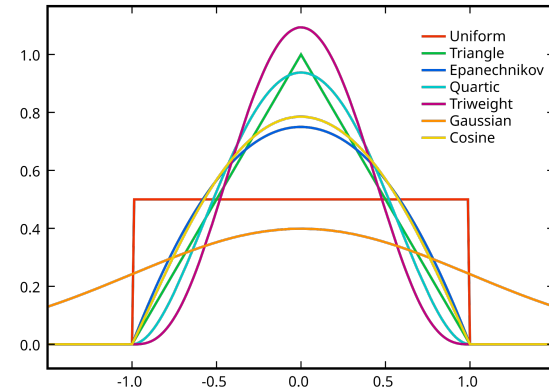
“Comparison of the histogram (left) and kernel density estimate (right) constructed using the same data. The six individual kernels are the red dashed curves, the kernel density estimate the blue curves. The data points are the rug plot on the horizontal axis.”

By Drleft at English Wikipedia, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=57332968>

KDE Underfitting vs Overfitting

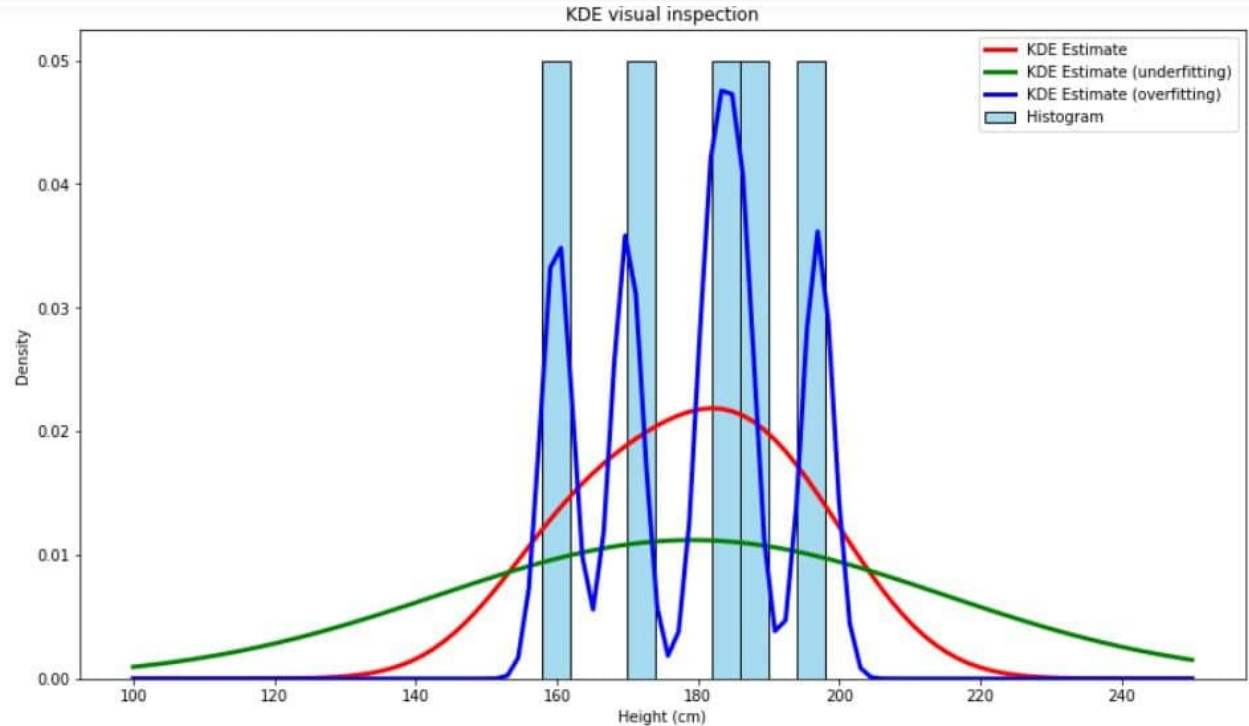


Different types of kernels



By Brian Amberg - Own work, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=5329895>

seaborn bandwidth kwarg:
bw_adjust: bigger = smoother ->
overfitting (lose detail), smaller =
volatile -> under fitting (lose trend)



<https://www.baeldung.com/cs/kde-tutorial>



kdeplot() (Univariate (1D))

Seaborn can plot a KDE plot with the **kdeplot()** function

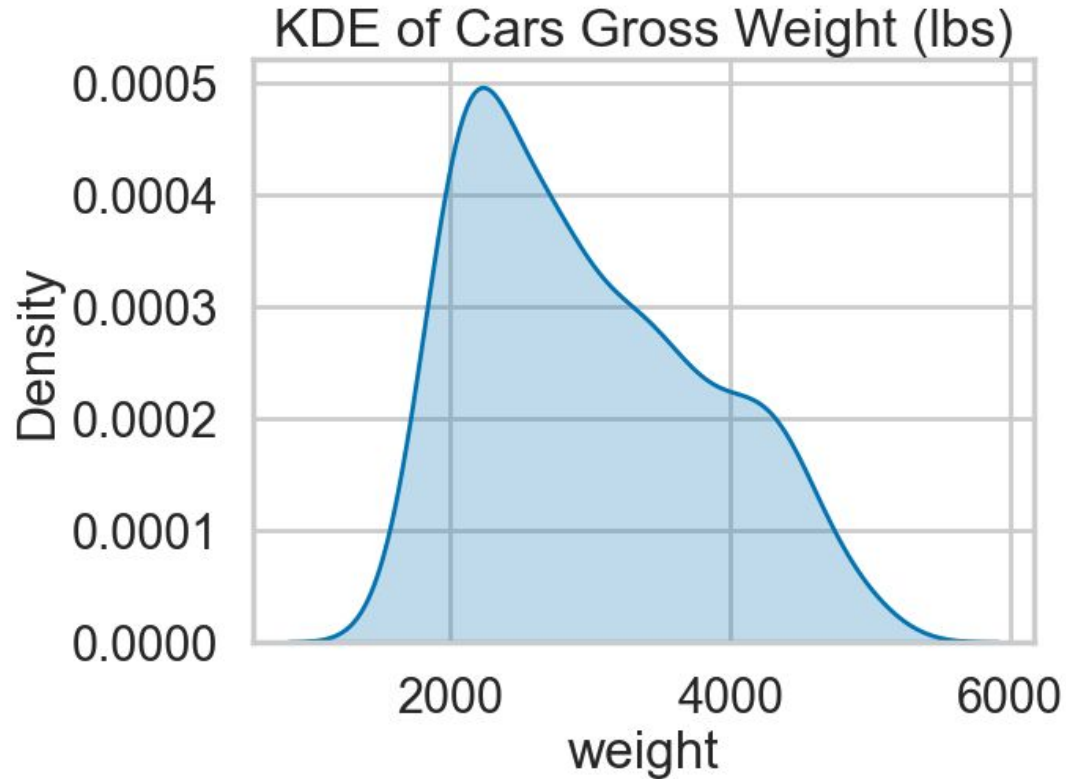
The first parameter is which feature/column/series you want to plot

bw_adjust controls the width of your kernel estimator

fill is similar to **shade** to shade the area under the curve

```
ax=sns.kdeplot(cars.weight,  
               bw_adjust=bandwidth_adjustment,  
               fill=True)  
ax.get_figure().savefig('kde.png',  
                        bbox_inches='tight');
```

kdeplot() (Univariate (1D)) output



Revisit histplot() and add a KDE overlay

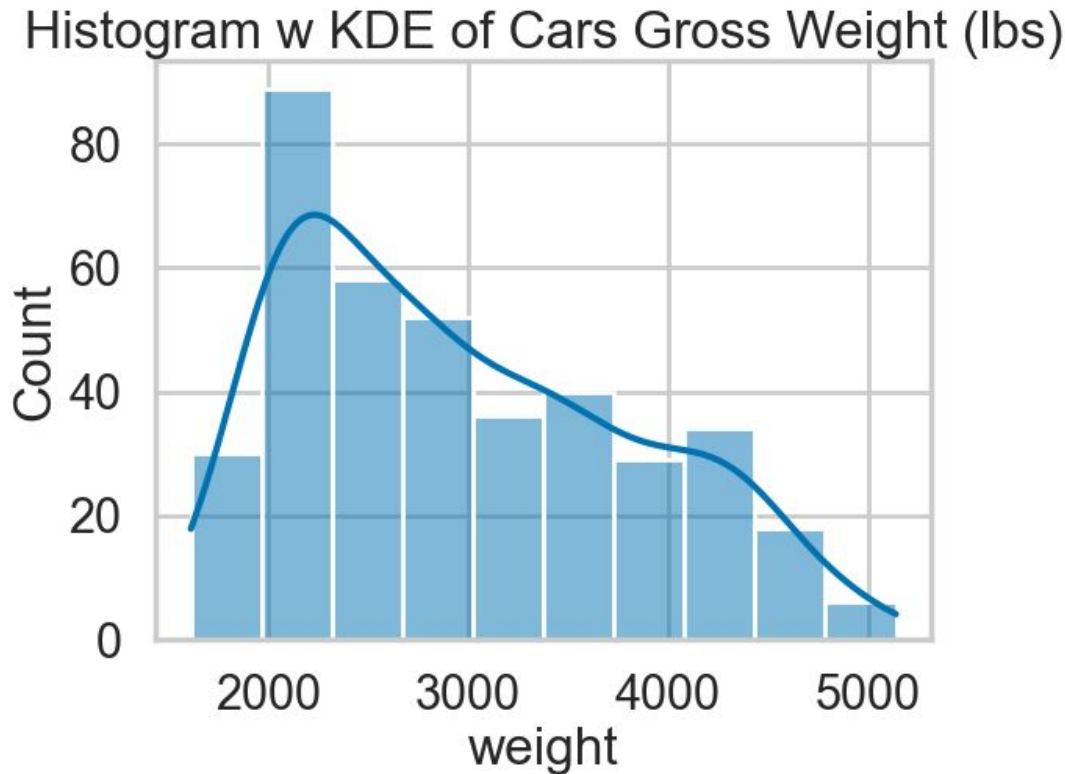


histplot() has an optional kwarg **kde** that defaults to False in the newer versions of seaborn but you can always override the default to True to overlay a KDE on top of your histogram.

The best of both worlds!

```
ax = sns.histplot(cars.weight,  
                 kde=True)  
ax.set_title('Histogram w KDE of  
Cars Gross Weight (lbs)')  
ax.get_figure().savefig('hist_KDE.  
png', bbox_inches='tight');
```

Revisit histplot() and add a KDE overlay output



2D (Bivariate) KDEs



seaborn can plot a 2D (bivariate) KDE plot, aka contour plot by specifying a 2nd variable as y.

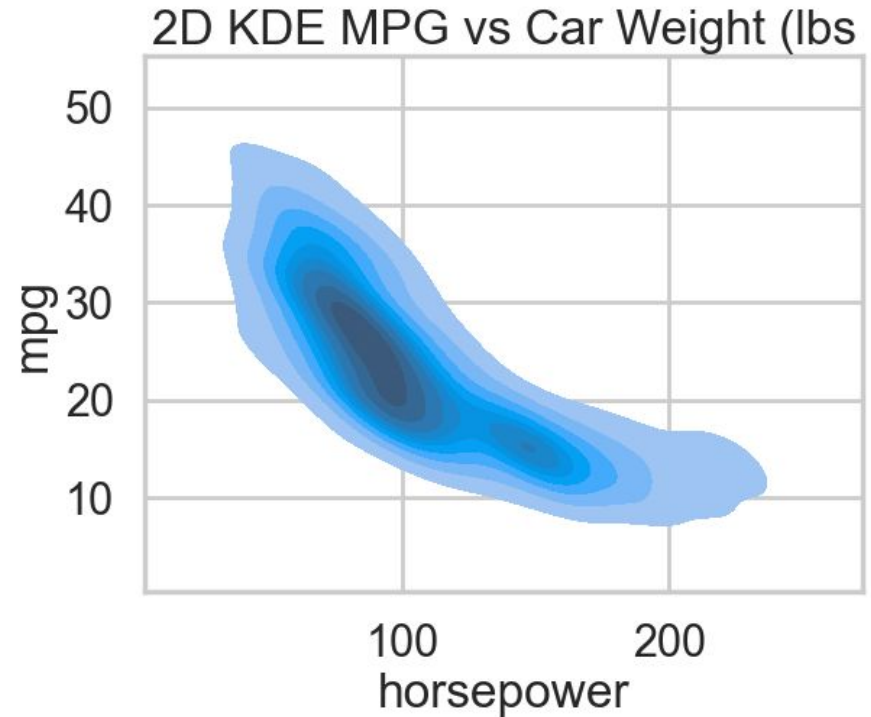
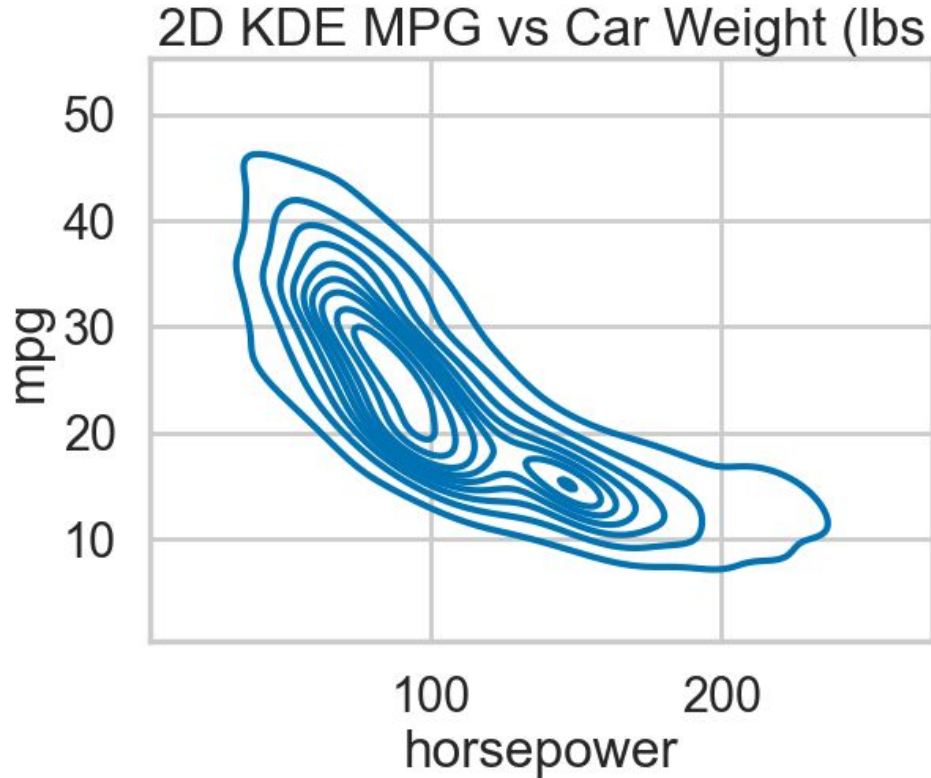
Most seaborn plots that take more than one variable now require explicitly specifying **x** and **y** parameters.

You can also optionally specify a pandas dataframe as the **data** kwarg which then allows you to specify just the pandas series name as a string.

The optional **fill** kwarg fills in the resulting contour map.

```
ax=sns.kdeplot(x=cars.weight,  
y=cars.mpg);  
  
# -OR-  
  
ax=sns.kdeplot(data=cars,  
x='horsepower', y='mpg',  
fill=True);  
  
ax.set_title('2D KDE MPG vs  
Car Weight (lbs)')  
  
ax.get_figure().savefig('kde2d  
_fill.png',  
bbox_inches='tight');
```

2D (Bivariate) KDEs output

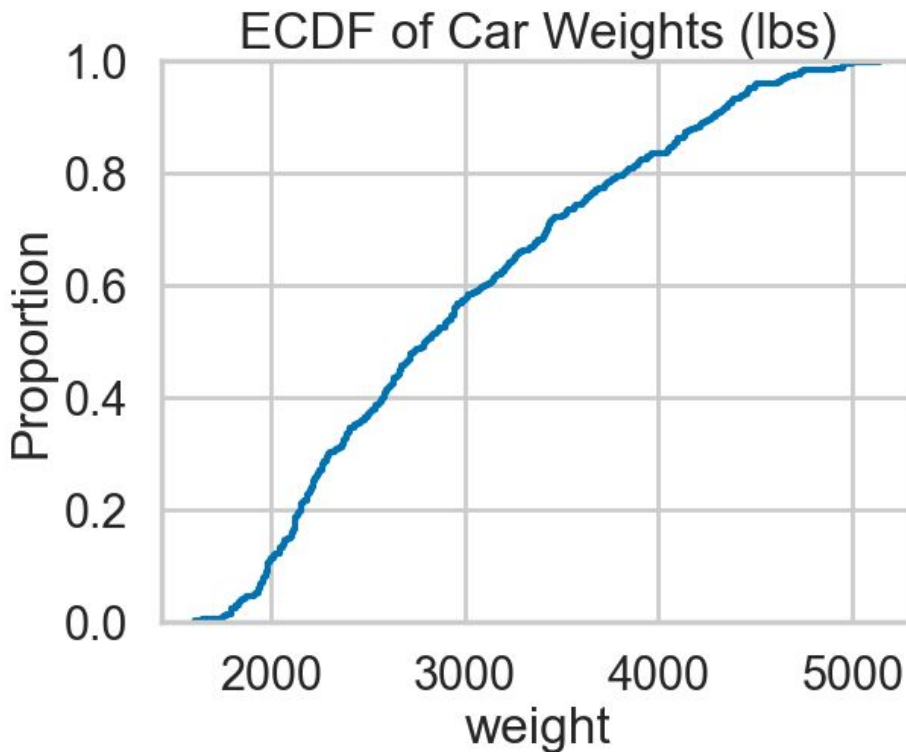


ECDF



An ECDF (Empirical Cumulative Distribution Function) shows the cumulative proportion distribution on the y-axis vs a feature on the x-axis.

From this chart, you can deduce that almost 60% of cars in this dataset are ~3,000 lbs or less.



ecdfplot()



Seaborn can plot an ECDF w/ the **ecdfplot()** function. Here we are passing a single pandas series, in this case, weight, to plot.

```
ax=sns.ecdfplot(cars.weight);  
ax.set_title('ECDF of Car  
Weights (lbs)')  
ax.get_figure().savefig('ecdf.  
png', bbox_inches='tight');
```

boxplot



When you call `sns.boxplot()`, it calls Matplotlib's internal functions (such as `ax.boxplot()`) to draw the actual plot elements.

```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([  
4,6,8])]
```

```
ax=sns.boxplot(x=cars.mpg,  
hue=cars.cylinders, orient='h',  
width=.2)
```

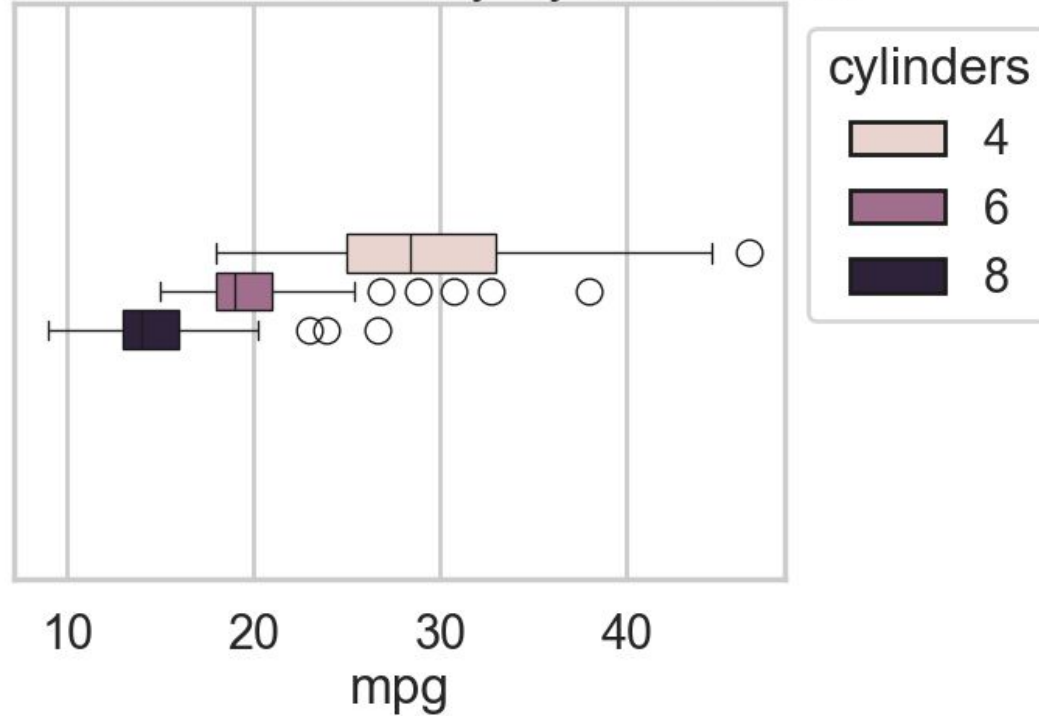
```
ax.set_title('Box Plot of Car  
MPG by Cylinder Count')
```

```
sns.move_legend(ax, "upper  
left", bbox_to_anchor=(1, 1));
```

boxplot output



Box Plot of Car MPG by Cylinder Count



boxplot :: pandas tabular data for labels (optional)



Verifying that boxplot **hue** behaves like pandas **groupby**.

```
#cars[cars.cylinders.isin([8])].mpg.describe()  
cars.groupby('cylinders').mpg.describe()
```

This is also an easy way to generate data to manually label a boxplot's milestones: median/50%, quartiles, outliers, etc.

violinplot



When you call `sns.violinplot()`,
makes a mirrored, symmetric
KDE plot w/ a boxplot in the
center.

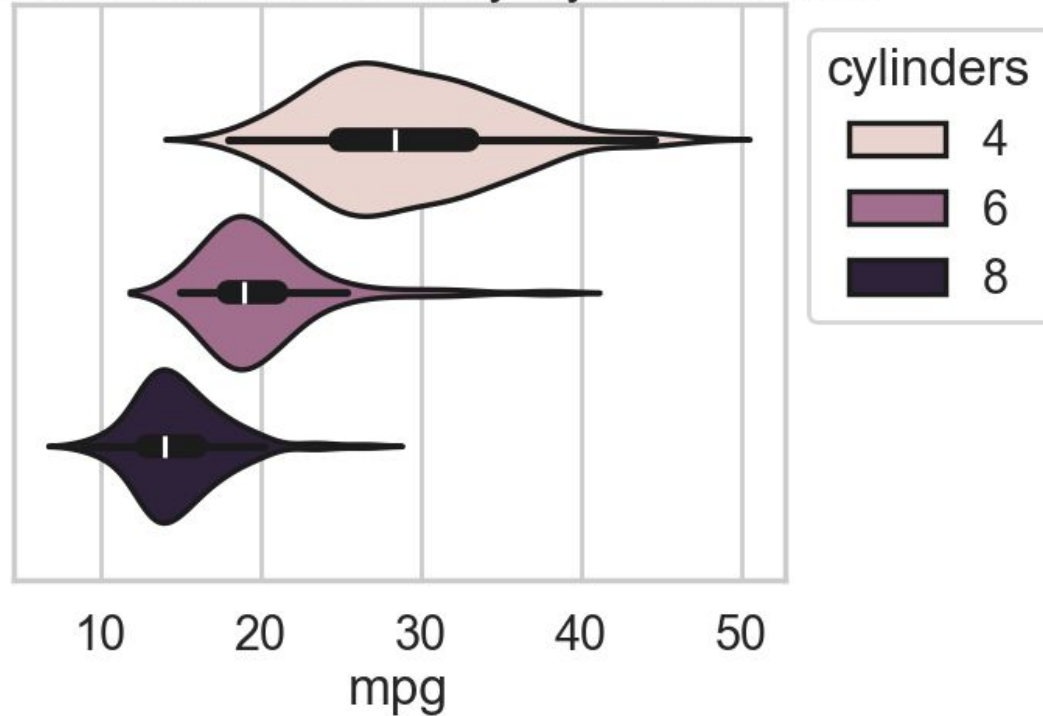
```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([  
4,6,8])] ]
```

```
ax=sns.violinplot(x=cars.mpg,  
hue=cars.cylinders)  
ax.set_title('Violin Plot of  
Car MPG by Cylinder Count')  
sns.move_legend(ax, "upper  
left", bbox_to_anchor=(1, 1));
```


violinplot output



Violin Plot of Car MPG by Cylinder Count



scatterplot

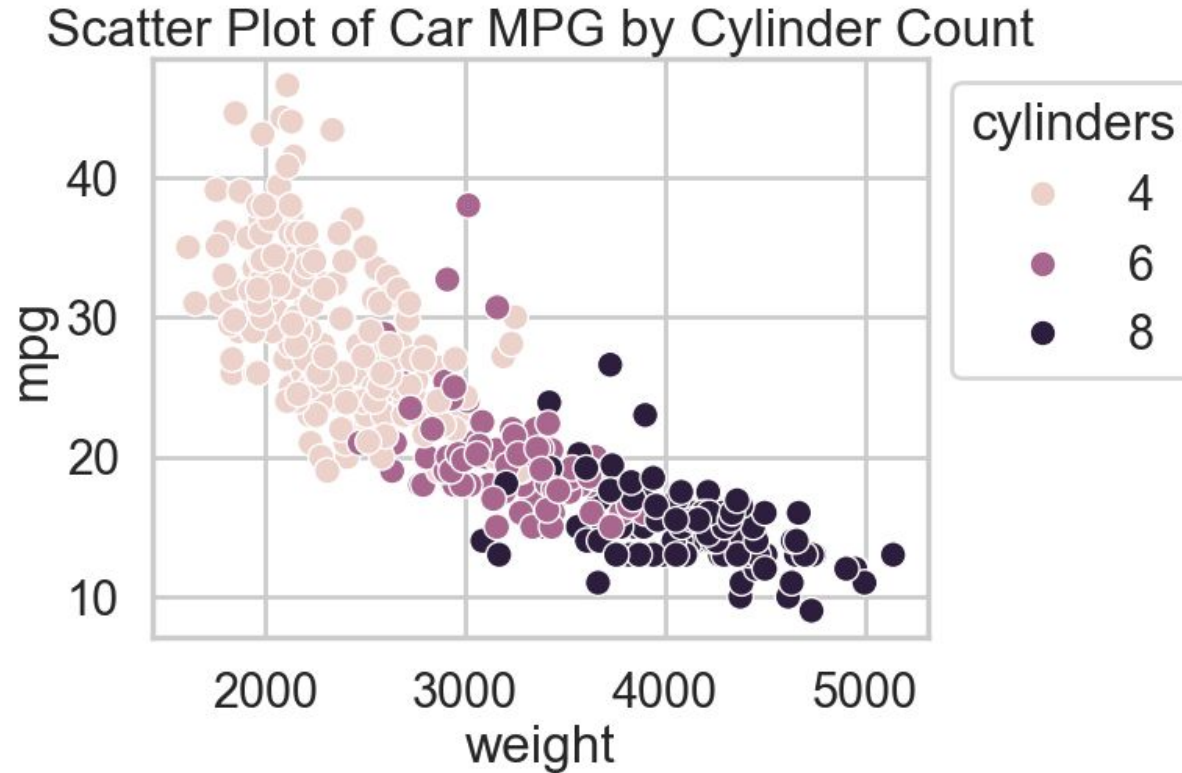


sns.scatterplot(), plots each row, individual data point.

```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([4,6  
,8])]
```

```
ax=sns.scatterplot(x=cars.weight,  
y=cars.mpg, hue=cars.cylinders)  
ax.set_title('Violin Plot of Car  
MPG by Cylinder Count')  
sns.move_legend(ax, "upper left",  
bbox_to_anchor=(1, 1));
```

scatterplot output



stripplot



Draw a categorical scatterplot using jitter to reduce overplotting.

A strip plot can be drawn on its own, but it is also a good complement to a box or violin plot in cases where you want to show all observations along with some representation of the underlying distribution.

<https://seaborn.pydata.org/generated/seaborn.stripplot.html>
[#seaborn.stripplot](#)

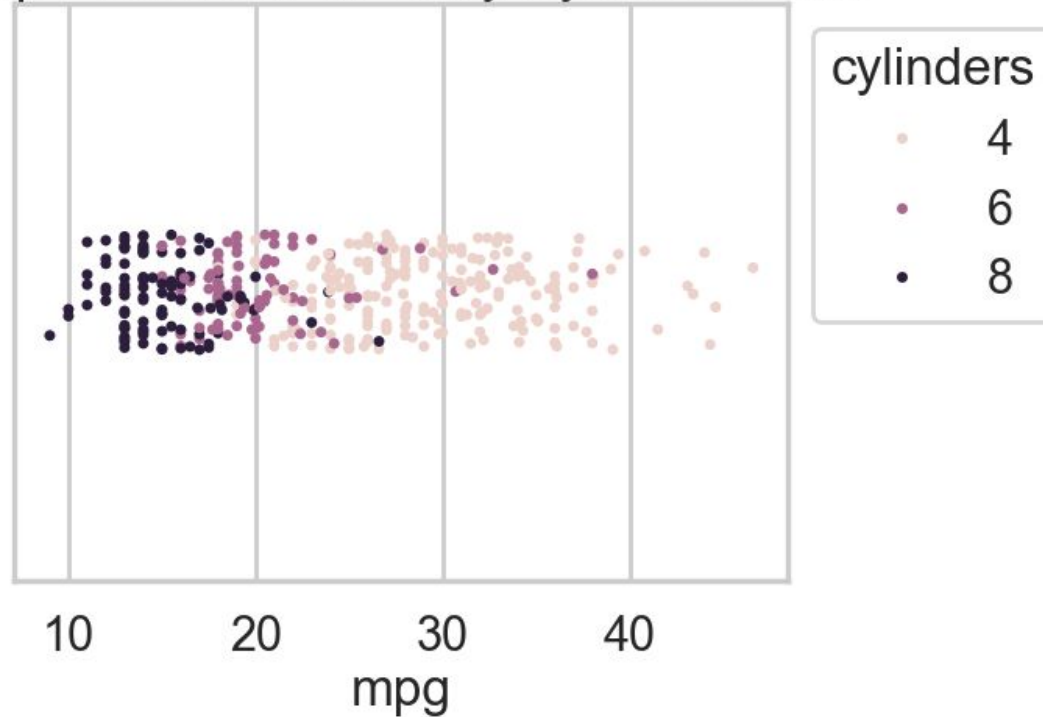
```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([  
4,6,8])]
```

```
ax=sns.stripplot(x=cars.mpg,  
hue=cars.cylinders)  
ax.set_title('Strip Plot Plot  
of Car MPG by Cylinder Count')  
sns.move_legend(ax, "upper  
left", bbox_to_anchor=(1, 1));
```

stripplot output



Strip Plot of Car MPG by Cylinder Count



swarmplot



Draw a categorical scatterplot with points adjusted to be non-overlapping.

This function is similar to `stripplot()`, but the points are adjusted (only along the categorical axis) so that they don't overlap. This gives a better representation of the distribution of values, but it does not scale well to large numbers of observations. This style of plot is sometimes called a “beeswarm”.

A swarm plot can be drawn on its own, but it is also a good complement to a box or violin plot in cases where you want to show all observations along with some representation of the underlying distribution.

<https://seaborn.pydata.org/generated/seaborn.swarmplot.html>

```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([  
4,6,8])] ]
```

```
ax=sns.swarmplot(x=cars.mpg,  
hue=cars.cylinders)
```

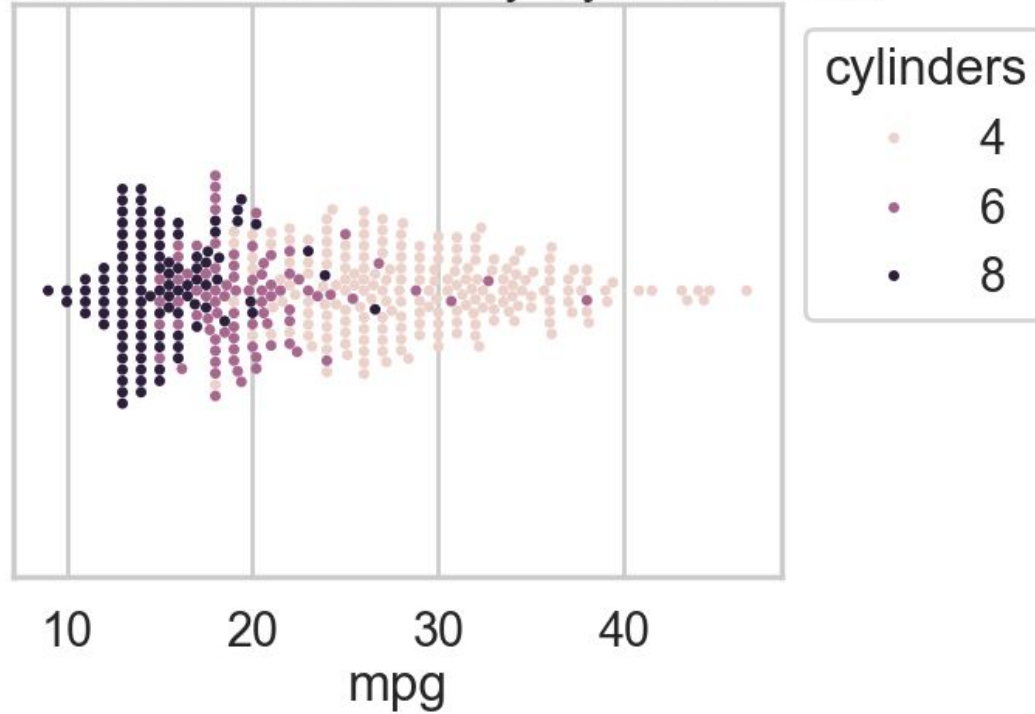
```
ax.set_title('Swarmplot Plot of  
Car MPG by Cylinder Count')
```

```
sns.move_legend(ax, "upper  
left", bbox_to_anchor=(1, 1));
```

swarmplot output



Swarm Plot of Car MPG by Cylinder Count



lineplot



Draw a line plot with possibility of several semantic groupings.

The relationship between x and y can be shown for different subsets of the data using the hue, size, and style parameters. These parameters control what visual semantics are used to identify the different subsets. It is possible to show up to three dimensions independently by using all three semantic types, but this style of plot can be hard to interpret and is often ineffective. Using redundant semantics (i.e. both hue and style for the same variable) can be helpful for making graphics more accessible.

<https://seaborn.pydata.org/generated/seaborn.lineplot.html>

```
cars.cylinders.value_counts()  
cars=cars[cars.cylinders.isin([  
4,6,8])] ]
```

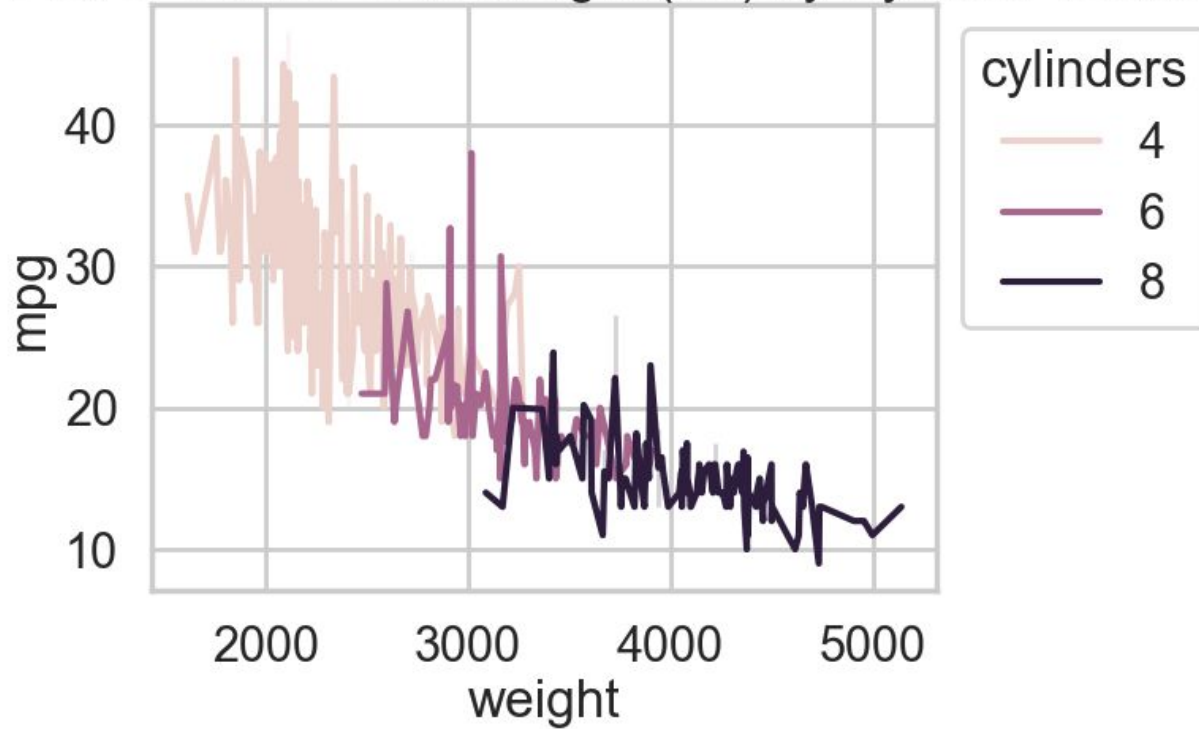
```
ax=sns.lineplot(x=cars.weight,  
y=cars.mpg, hue=cars.cylinders)  
ax.set_title('Line Plot of Car  
MPG vs Weight (lbs) by Cylinder  
Count')
```

```
sns.move_legend(ax, "upper  
left", bbox_to_anchor=(1, 1));
```


lineplot output



Line Plot of Car MPG vs Weight (lbs) by Cylinder Count



lineplot w/ Confidence Intervals



Passing the entire dataset in long-form mode will aggregate over repeated values (each year) to show the mean and 95% confidence interval:

<https://seaborn.pydata.org/generated/seaborn.lineplot.html>

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121

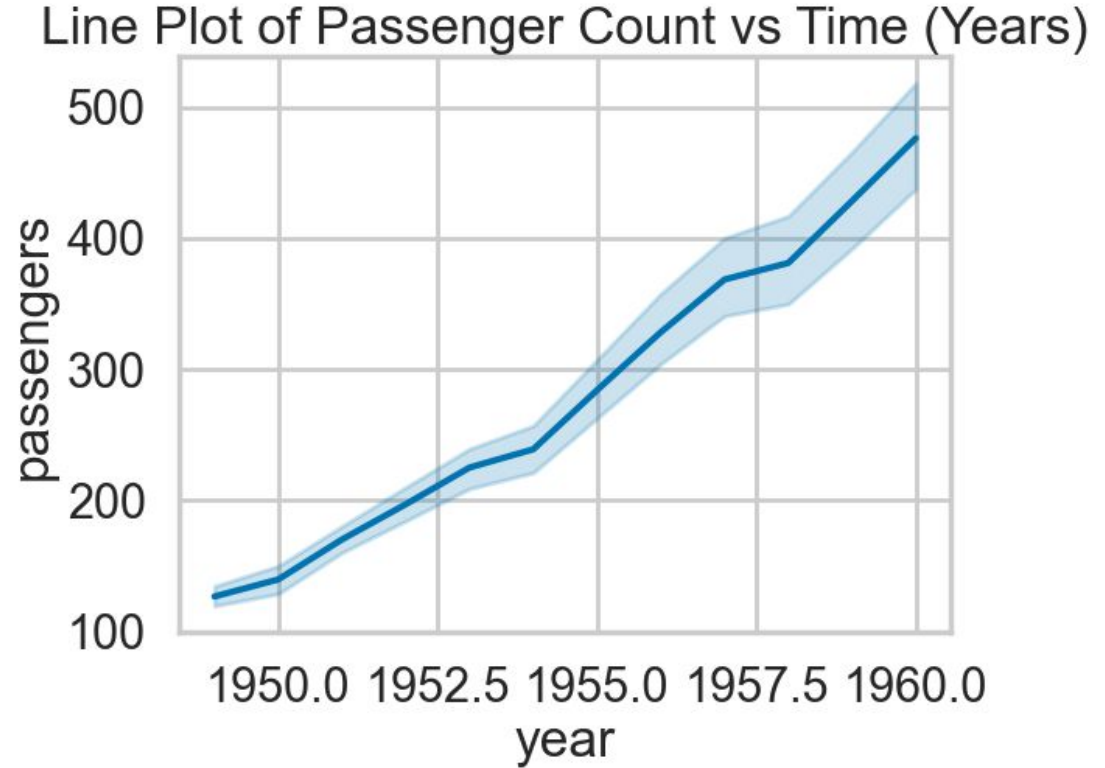
```
flights =  
sns.load_dataset("flights")  
flights.head()
```

```
ax=sns.lineplot(data=flights,  
x="year", y="passengers")  
ax.set_title('Line Plot of  
Passenger Count vs Time  
(Years)')
```

lineplot w/ Confidence Interval output

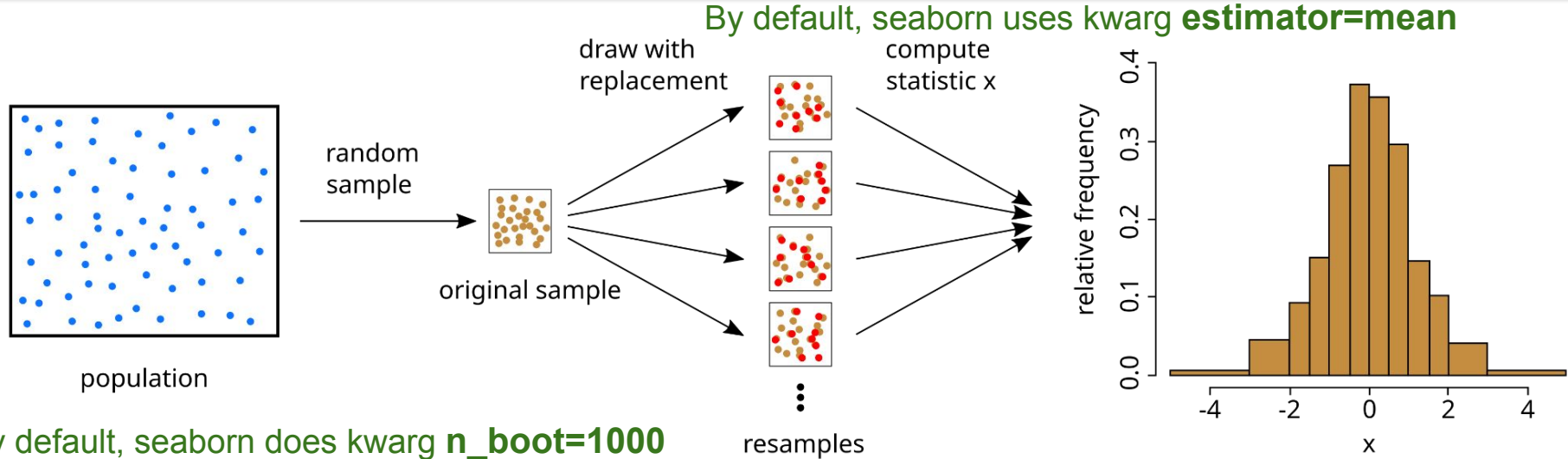


	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121





Bootstrapping Confidence Intervals



By Biggerj1, Marsupilami - * File:Thist german.png, Autor: MM-Stat<https://postimg.cc/MffYNyKZ>, Autor Biggerj1, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=135426288>

Seaborn then selects upper and lower bounds so that kwarg **ci=95** % of these resampled means() fit in that range, this is the confidence interval

regplot



Plot data and a linear regression model fit.

There are a number of mutually exclusive options for estimating the regression model. See the [tutorial](#) for more information.

Fit a **robust** regression to downweight the influence of outliers:

<https://seaborn.pydata.org/generated/seaborn.regplot.html>

The **scatter_kws** kwarg lets you specify optional key worded arguments through to the matplotlib scatter() plot call.

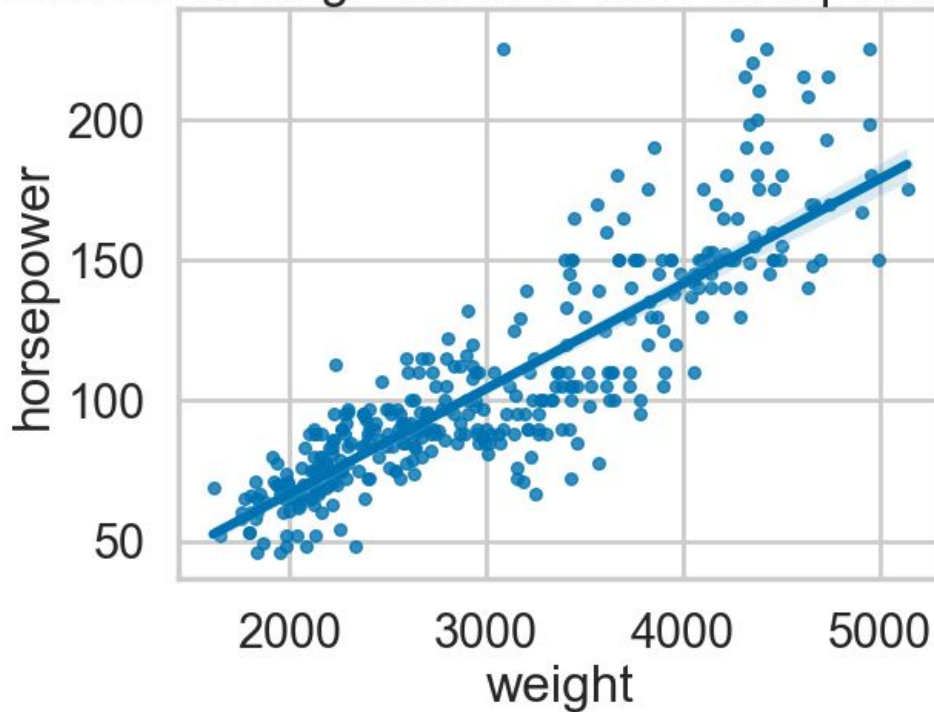
```
ax=sns.regplot(data=cars,  
y="horsepower", x="weight",  
robust=True, scatter_kws={"s":  
30})
```

```
ax.set_title('Robust Linear  
Regression of Car Horsepower vs  
Weight')
```

regplot w/ robust (downplayed outliers) linreg



Robust Linear Regression of Car Horsepower vs Weight



heatmap



Plot rectangular data as a color-encoded matrix.

It's often used to visualize correlations of every feature vs every feature of a dataset; Pandas can create this in tabular form w/ its **.corr()** method.

The **cmap** kwarg lets you specify a colormap. '**RdBu**' spans the gamut from Red to Blue; Red being a negative correlation (inversely proportional) and Blue being a positive correlation (directly proportional).

The **annot** kwarg adds annotations to each square w/ the correlation numerical value.

The **annot_kws** kwarg lets you specify things like font size and weight.

<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

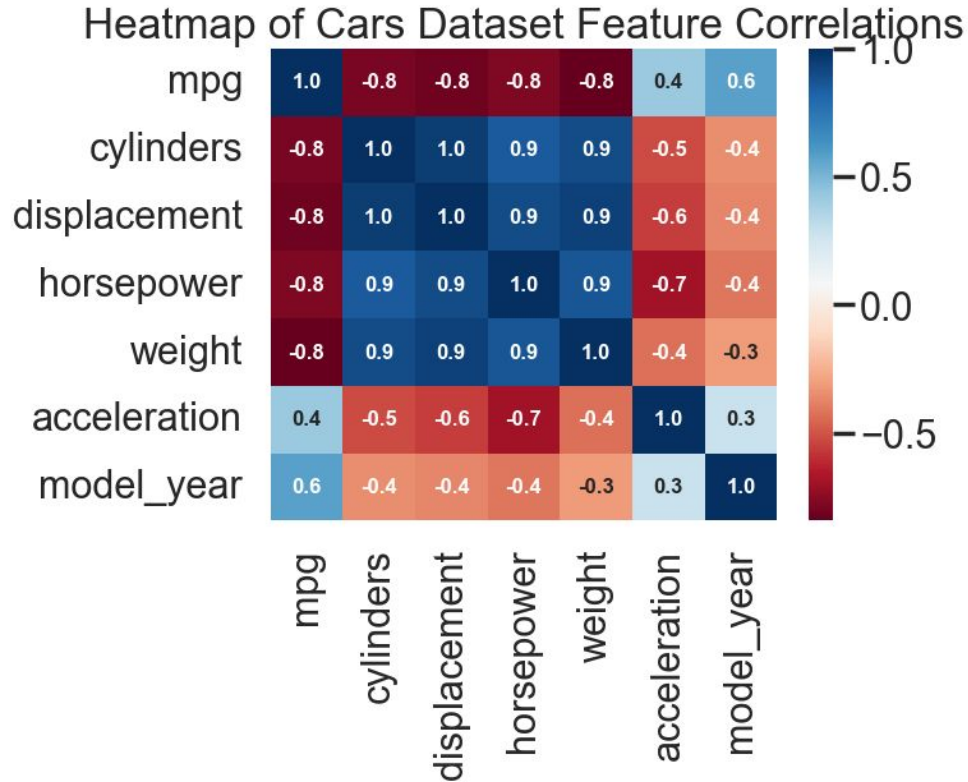
```
cars_corr=cars.corr(numeric_only=True)
```

```
ax=sns.heatmap(cars_corr, cmap='RdBu',  
annot=True, fmt='0.1f',
```

```
    annot_kws={  
        'fontsize': 12,  
        'fontweight': 'bold'}  
    );
```

```
ax.set_title('Heatmap of Cars Dataset  
Feature Correlations')
```

heatmap output



clustermap



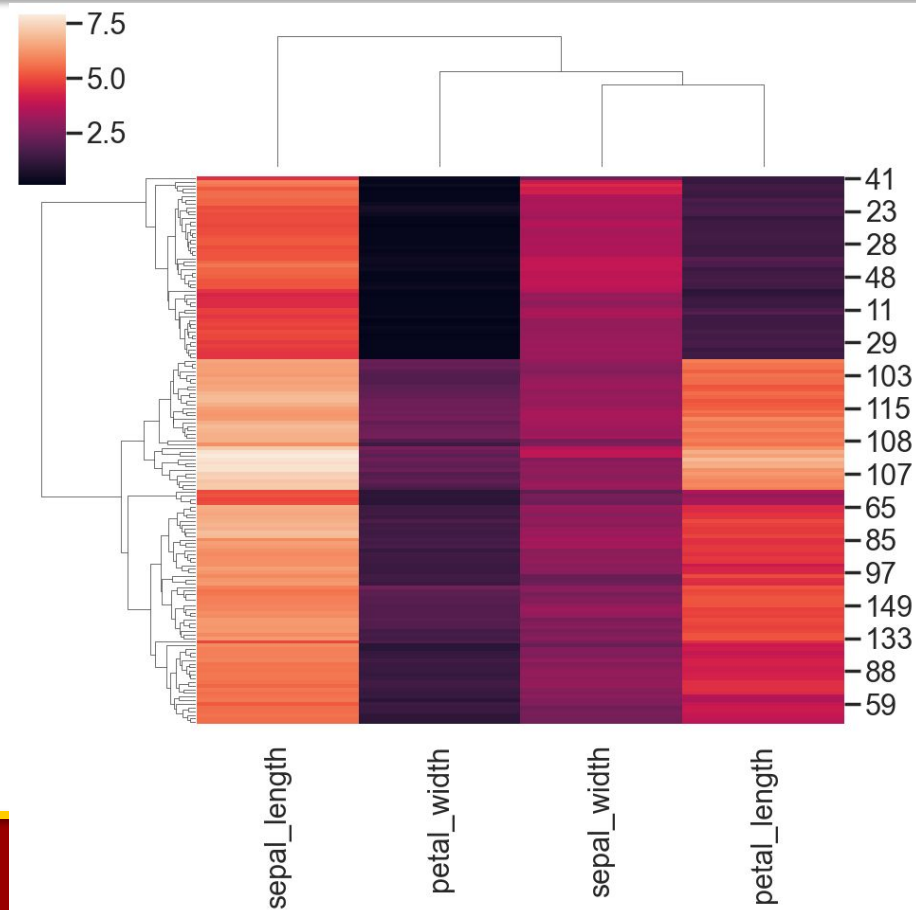
Plot a matrix dataset as a hierarchically-clustered heatmap.

This function requires scipy to be available.

<https://seaborn.pydata.org/generated/seaborn.clustermap.html>

```
iris = sns.load_dataset("iris")
_ = iris.pop("species")
sns.clustermap(iris)
```

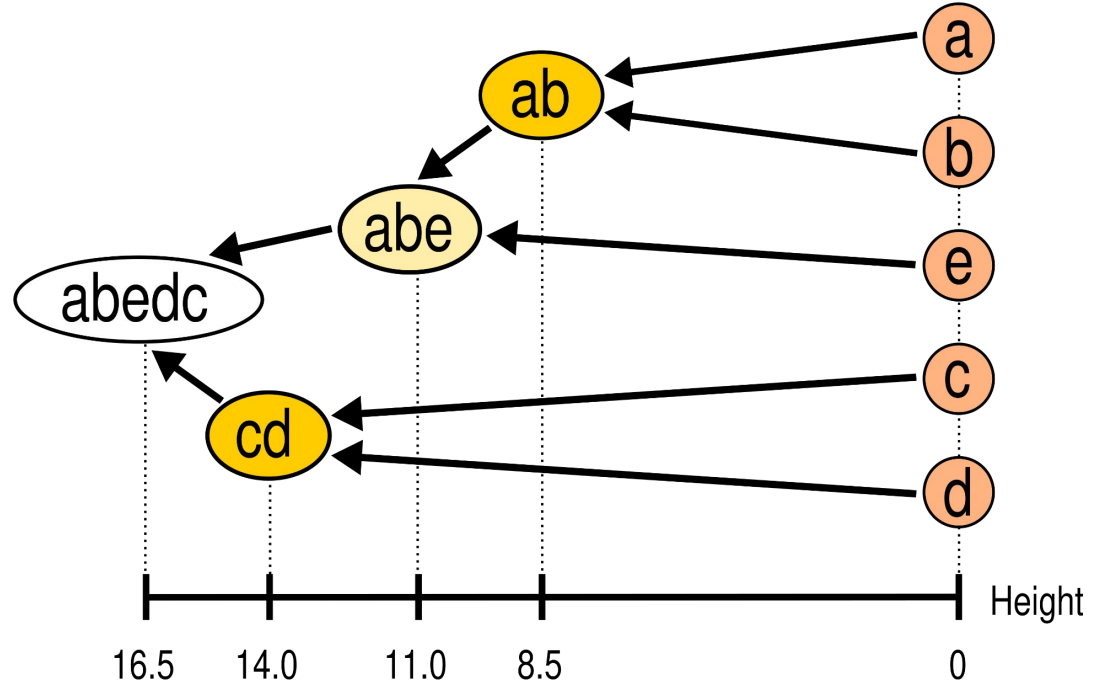
clustermat output



dendrogram



Dendograms show hierarchical clustering/similarity



By This picture is a work by Emmanuel Douzery. Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=73759227>

barplot



Show point estimates and errors as rectangular bars.

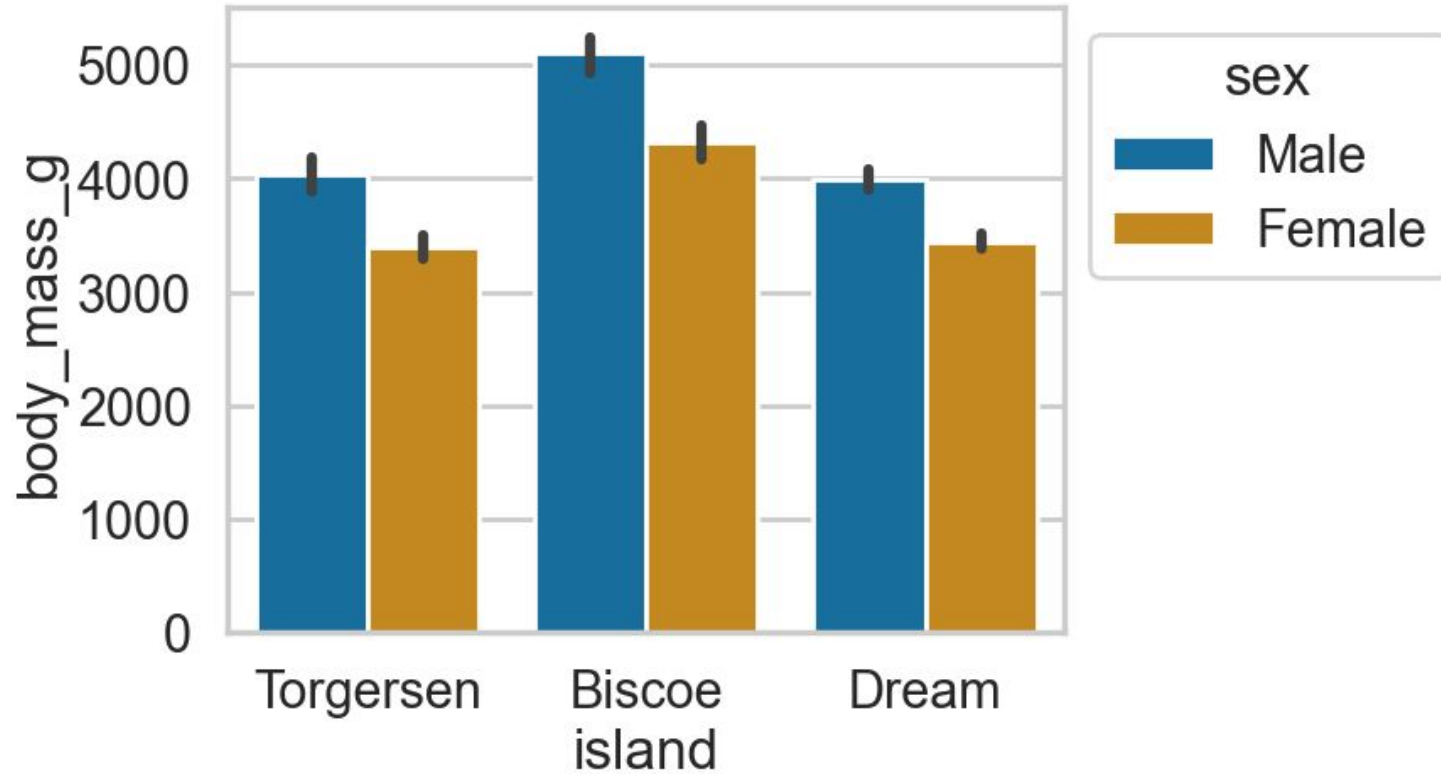
A bar plot represents an aggregate or statistical estimate for a numeric variable with the height of each rectangle and indicates the uncertainty around that estimate using an error bar. Bar plots include 0 in the axis range, and they are a good choice when 0 is a meaningful value for the variable to take.

Note: The error bar at the top shows the 95% confidence interval of mean by default.

<https://seaborn.pydata.org/generated/seaborn.barplot.html>

```
penguins=sns.load_dataset('penguins')
ax=sns.barplot(penguins, x="island",
y="body_mass_g", hue="sex")
ax.set_title('Penguin Body Mass (g) by
Sex')
sns.move_legend(ax, "upper left",
bbox_to_anchor=(1, 1));
```

barplot output



countplot



Show the counts of observations in each categorical bin using bars.

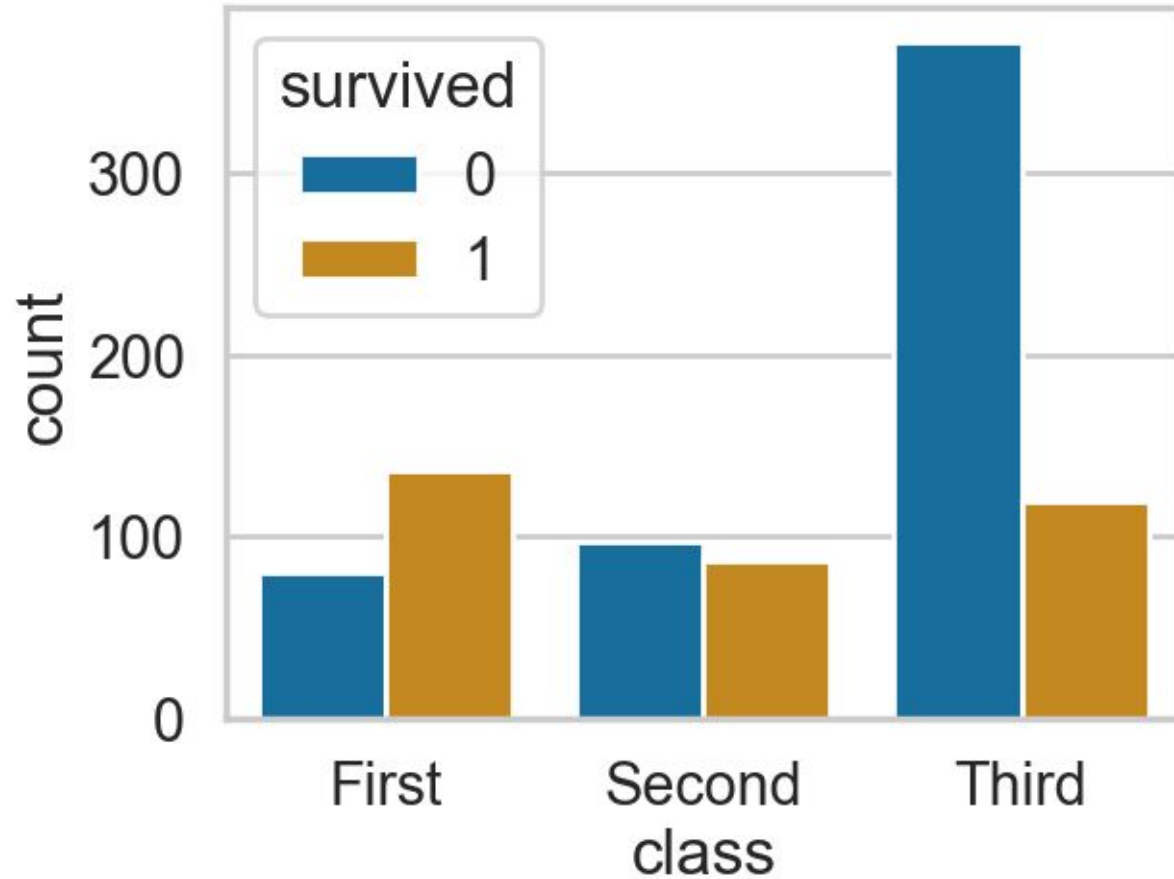
A count plot can be thought of as a histogram across a categorical, instead of quantitative, variable. The basic API and options are identical to those for `barplot()`, so you can compare counts across nested variables.

Note that `histplot()` function offers similar functionality with additional features (e.g. bar stacking), although its default behavior is somewhat different.

<https://seaborn.pydata.org/generated/seaborn.countplot.html>
!

```
titanic=sns.load_dataset('titanic')  
ax=sns.countplot(titanic, x="class",  
hue="survived")
```

countplot output



pairplot



Plot pairwise relationships in a dataset.

By default, this function will create a grid of Axes such that each numeric variable in `data` will be shared across the y-axes across a single row and the x-axes across a single column. The diagonal plots are treated differently: a univariate distribution plot is drawn to show the marginal distribution of the data in each column.

It is also possible to show a subset of variables or plot different variables on the rows and columns.

This is a high-level interface for `PairGrid` that is intended to make it easy to draw a few common styles. You should use `PairGrid` directly if you need more flexibility.

<https://seaborn.pydata.org/generated/seaborn.pairplot.html>

```
pg=sns.pairplot(data=cars,
hue='origin')

pg.figure.suptitle('Pair Plot of Cars
Dataset Features')

sns.move_legend(pg, loc="upper left",
bbox_to_anchor=(1.0, 1.0))

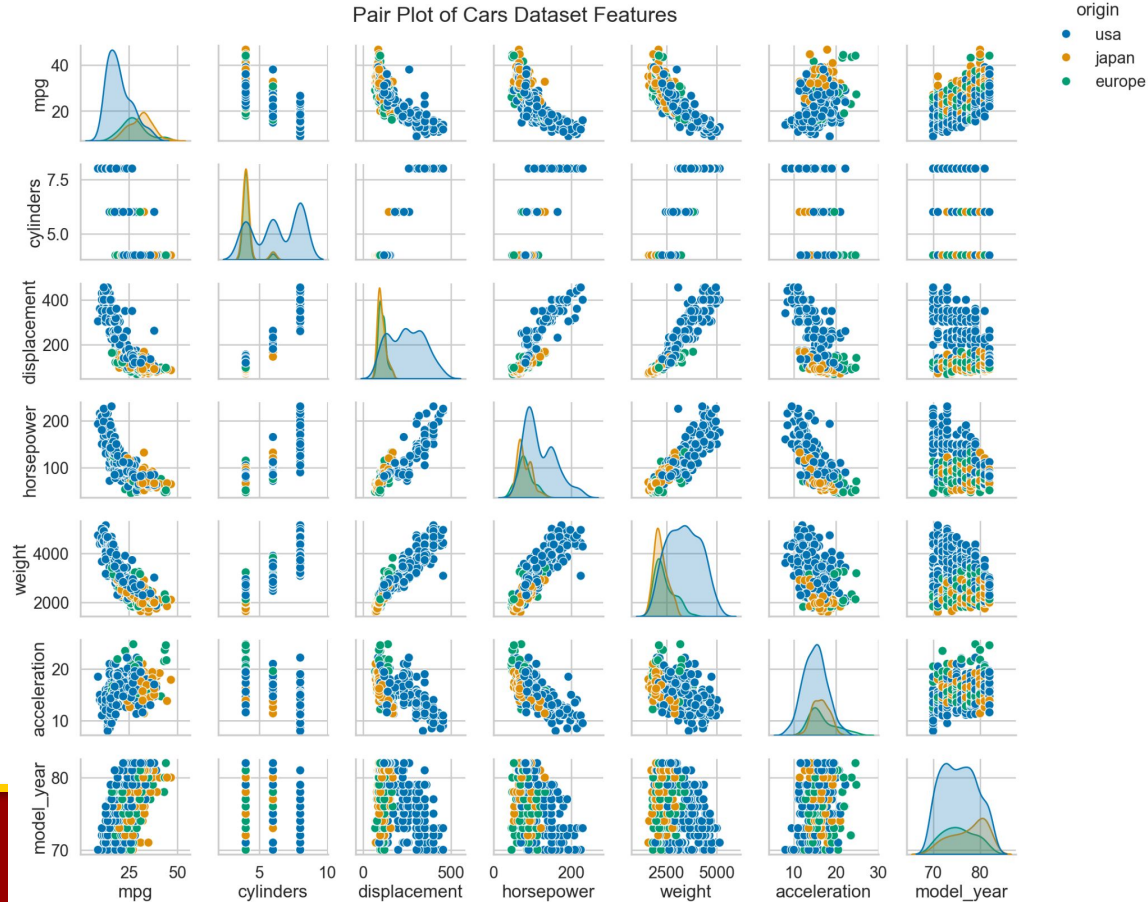
pg.figure.tight_layout()

pg.savefig('pairplot.png')
```


pairplot output



Pair Plot of Cars Dataset Features



jointplot



Draw a plot of two variables with bivariate and univariate graphs.

This function provides a convenient interface to the `JointGrid` class, with several canned plot kinds. This is intended to be a fairly lightweight wrapper; if you need more flexibility, you should use `JointGrid` directly.

<https://seaborn.pydata.org/generated/seaborn.jointplot.html>

```
jg=sns.jointplot(data=cars, x='horsepower',  
y='mpg')
```

```
jg.figure.suptitle('Joint Plot of Car MPG vs  
Horse Power')
```

```
jg.savefig('jointplot.png')
```

```
jg=sns.jointplot(data=cars, x='horsepower',  
y='mpg', kind='kde', hue='origin')
```

```
jg.figure.suptitle('KDE Joint Plot of Car MPG  
vs Horse Power')
```

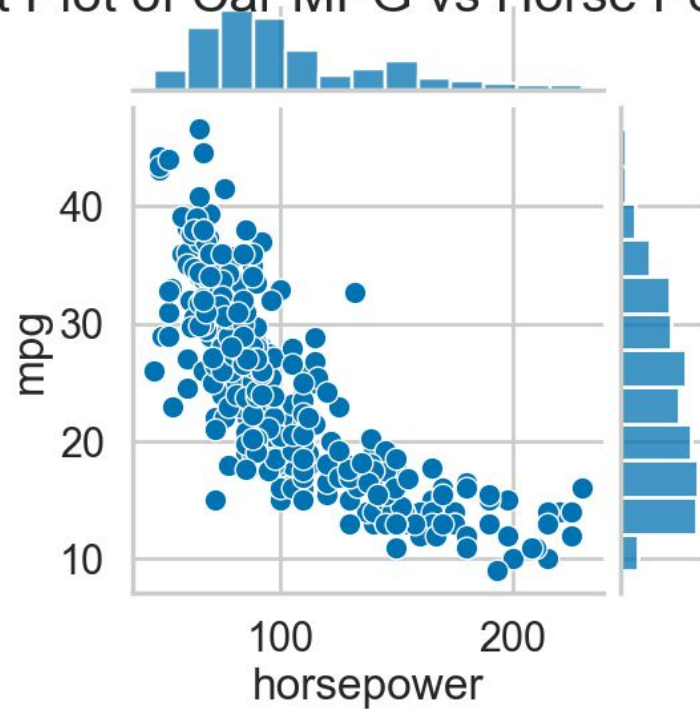
```
sns.move_legend(jg.ax_joint, loc="upper left",  
bbox_to_anchor=(1.2, 1.0))
```

```
jg.savefig('jointplot_KDE.png')
```

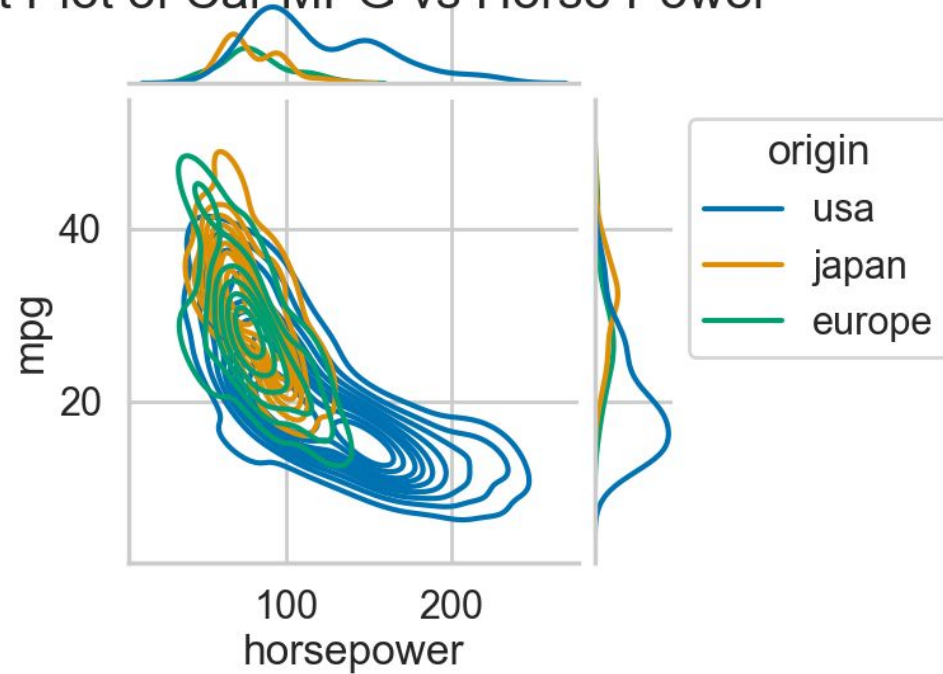


jointplot output

Joint Plot of Car MPG vs Horse Power



Joint Plot of Car MPG vs Horse Power



Resources



- [seaborn's own tutorial](#)
- [Dr. Kimberly Fessel's Intro to seaborn YT Playlist](#)

Questions?

